

PruneRAG: Confidence-Guided Query Decomposition Trees for Efficient Retrieval-Augmented Generation

Shuguang Jiao
Harbin Institute of
Technology, Shenzhen
Shenzhen, China
24S151028@stu.hit.edu.cn

Xinyu Xiao
Harbin Institute of
Technology, Shenzhen
Shenzhen, China
23b951021@stu.hit.edu.cn

Yunfan Wei
South China University of
Technology
Guangzhou, China
msyunfan@mail.scut.edu.cn

Shuhan Qi*
Harbin Institute of
Technology, Shenzhen and
Leanplans
Shenzhen, China
shuhanqi@cs.hitsz.edu.cn

Chengkai Huang*
Macquarie University and
The University of New
South Wales
Sydney, Australia
chengkai.huang1@unsw.edu.au

Quan Z. Sheng
Macquarie University
Sydney, Australia
michael.sheng@mq.edu.au

Lina Yao
The University of New
South Wales and CSIRO's
Data61
Sydney, Australia
lina.yao@unsw.edu.au

Abstract

Retrieval-augmented generation (RAG) has become a powerful framework for enhancing large language models in knowledge-intensive and reasoning tasks. However, as reasoning chains deepen or search trees expand, RAG systems often face two persistent failures: evidence forgetting, where retrieved knowledge is not effectively used, and inefficiency, caused by uncontrolled query expansions and redundant retrieval. These issues reveal a critical gap between retrieval and evidence utilization in current RAG architectures. We propose PruneRAG, a confidence-guided query decomposition framework that builds a structured query decomposition tree to perform stable and efficient reasoning. PruneRAG introduces three key mechanisms: adaptive node expansion that regulates tree width and depth, confidence-guided decisions that accept reliable answers and prune uncertain branches, and fine-grained retrieval that extracts entity-level anchors to improve retrieval precision. Together, these components preserve salient evidence throughout multi-hop reasoning while significantly reducing retrieval overhead. To better analyze evidence misuse, we define the Evidence Forgetting Rate as a metric to quantify cases where golden evidence is retrieved but not correctly used. Extensive experiments across various multi-hop QA benchmarks show that PruneRAG achieves superior accuracy and efficiency over state-of-the-art baselines. The code is publicly available.¹

CCS Concepts

• Information systems → Information retrieval.

*Corresponding authors.

¹<https://github.com/Fdioa/PruneRAG>



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792103>

Keywords

Retrieval-Augmented Generation, Large Language Model, Tree-based RAG

ACM Reference Format:

Shuguang Jiao, Xinyu Xiao, Yunfan Wei, Shuhan Qi, Chengkai Huang, Quan Z. Sheng, and Lina Yao. 2026. PruneRAG: Confidence-Guided Query Decomposition Trees for Efficient Retrieval-Augmented Generation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792103>

1 Introduction

Retrieval-augmented generation (RAG) provides an effective and widely adopted paradigm for enhancing Large Language Models (LLMs) on knowledge-intensive and reasoning tasks [6, 8, 10, 11, 19, 31]. In open-domain QA and multi-hop fact verification, LLMs need to integrate evidence across multiple documents, often via iterative retrieval and reasoning. A common strategy is to decompose complex questions into sub-questions and perform step-by-step retrieval and generation [2, 20].

However, as reasoning chains grow longer or search trees deepen, two common failures emerge: *evidence forgetting*, where the model retrieves key evidence but fails to leverage it in later steps, and *inefficiency*, where uncontrolled expansions and redundant retrievals lead to high latency and cost. Figure 1(a) illustrates the forgetting phenomenon on HotpotQA, where golden evidence is retrieved early but later overshadowed by accumulated context. These failures reveal a persistent and critical gap between retrieval and effective evidence utilization in current RAG systems.

Chain-based RAG extends sequential reasoning to retrieval-augmented settings [3, 40, 50], decomposing a complex question into sub-questions that are solved step by step while dynamically invoking external knowledge [5]. As question complexity increases, reasoning chains lengthen and crucial early evidence is gradually diluted or forgotten over successive steps [43], destabilizing the final answers. Although on-demand retrieval reduces part of the

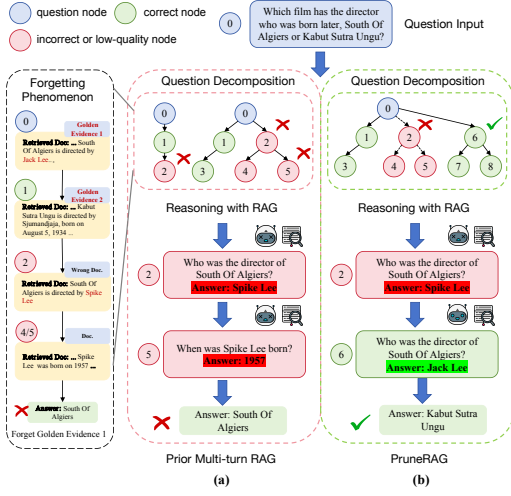


Figure 1: The left Figure (a) illustrates Multi-turn RAG’s sequential decomposition, showing how a low-quality or incorrect answer from an intermediate sub-question propagates, leading to an incorrect final result. The right Figure (b) demonstrates our PruneRAG method. By rejecting the low-quality answer and reflecting to generate a correct, high-quality answer, PruneRAG prevents downward error propagation and successfully arrives at the correct final answer.

redundancy, the lack of strong filtering and selection allows irrelevant content to be introduced, further exacerbating the forgetting of key information [21, 32].

Tree-based RAG seeks robustness by exploring multiple reasoning paths and selecting a promising trajectory [13, 23, 27, 30, 34]. For example, ConTReGen hierarchically expands semantic facets of a query [27], and RAG-Star employs Monte Carlo tree search to navigate paths [13]. Yet as tree depth increases, these methods struggle to maintain effective transmission of salient evidence: key facts are mixed with redundant context, and the absence of disciplined node-expansion and pruning yields many low-value nodes, driving up inference cost and obscuring crucial information.

Across both families, a common weakness is the lack of a *confidence-guided control mechanism* that decides when to stop answering, when to split a query, and when to refine retrieval. Without reliable control, reasoning tends to over-expand or drift, resulting in a systemic pattern of “retrieved but unused” evidence.

We propose PruneRAG, a tree-structured RAG framework for stable and efficient parallel reasoning as shown in Figure 1(b). PruneRAG iteratively constructs a query decomposition tree that breaks a complex question into independently solvable sub-problems, then aggregates child information during a backtracking phase to preserve and transmit key evidence throughout the reasoning process. To further improve efficiency and robustness, PruneRAG integrates three mechanisms: (i) *adaptive node expansion*, which regulates width and depth by deciding expansion based on the current query and retrieved context; (ii) *confidence-guided decisions*, which estimate answer confidence from token-level probabilities to accept high-confidence answers early and suppress low-confidence branches that cause drift; and (iii) *fine-grained retrieval*, which extracts key entities when a query cannot be further decomposed,

enabling precise retrieval that counteracts noise and context mismatch.

To diagnose the core failure of evidence misuse, we introduce the **Evidence Forgetting Rate (EFR)**, which measures the proportion of cases where golden evidence has been retrieved but the answer is still incorrect. EFR complements standard end-task accuracy by directly targeting evidence utilization; we provide the formal definition in Section 4.2.2.

The main contributions of this paper are as follows:

- We propose PruneRAG, a confidence-guided query decomposition tree for RAG that unifies answering, decomposition, and fine-grained retrieval, enabling structured and parallelizable reasoning while mitigating key-evidence forgetting.
- We design an adaptive expansion and pruning mechanism guided by answer probabilities, improving efficiency and reducing both redundancy and path drift.
- We introduce the evidence forgetting rate (EFR) as a diagnostic metric of evidence usability, and show that PruneRAG consistently reduces EFR across datasets, with an average reduction of 20.8% compared to mainstream multi-retrieval baselines, demonstrating its effectiveness in mitigating evidence forgetting.
- We conduct evaluations on three multi-hop QA datasets. Experimental results show that PruneRAG improves F1 score by 5.45% on average over the strongest baseline, while achieving a 4.9× speedup compared to mainstream multi-retrieval baselines.

2 Related Work

Retrieval-Augmented Reasoning. Retrieval-augmented generation (RAG) has become a key paradigm for improving the factuality of large language models (LLMs). Representative methods such as REALM [8], RAG [19], and DPR [16] incorporate document retrieval during inference to compensate for limited parametric knowledge, showing strong performance across QA, fact verification, and knowledge-intensive tasks. Meanwhile, Chain-of-Thought (CoT) [44] prompting enhances reasoning interpretability and stability by encouraging intermediate steps. Recent efforts combine CoT with RAG [12, 33]: ReAct [50] alternates between reasoning and retrieval in a closed loop, Self-RAG [1] generates intermediate queries to retrieve supporting evidence, and Search-o1 [20] selects and integrates relevant content at a fine-grained level. Although chain-based methods simplify the reasoning process, their inherently linear structure makes it difficult to support parallel processing of multiple reasoning paths. Moreover, since information is passed in a linearly accumulated manner between steps, critical information retrieved in earlier stages tends to be naturally diluted by later context, leading to information forgetting, reasoning path drift, and redundant retrieval—ultimately limiting the depth and efficiency of reasoning in complex tasks [35, 36].

Tree-based Retrieval-Augmented Generation. In existing tree-based Retrieval-Augmented Generation methods [2, 4, 23, 34, 37, 38, 46, 49, 51], the tree structure mainly serves as a tool for information expansion and path search. ConTReGen’s [27] tree structure achieves retrieval expansion by hierarchically exploring multiple semantic facets of queries, but it does not implement multi-step reasoning to solve problems and only expands retrieval knowledge; RAG-Star [13] leverages Monte Carlo Tree Search to

explore reasoning paths, but its external knowledge is only used to verify intermediate reasoning results and does not participate in the generation of reasoning nodes. Essentially, it is an optimal search of linear reasoning paths and cannot support independent parallel reasoning. Moreover, due to the lack of efficient structural control mechanisms, existing methods tend to generate a large number of redundant nodes and retrieved documents when handling complex queries, leading to prolonged reasoning processes and significantly increased computational overhead [28, 29], which severely limits their applicability in high-efficiency reasoning scenarios.

However, existing methods do not explicitly address the issue of evidence forgetting, where retrieved golden information fails to be effectively utilized in subsequent reasoning steps.

3 PruneRAG

In this section, we introduce PruneRAG, which integrates three key mechanisms: adaptive query decomposition (Section 3.2.1) for structured problem decomposition, confidence-guided pruning (Section 3.2.2) for filtering low-confidence branches and avoiding redundancy, and fine-grained retrieval (Section 3.2.3) for precise evidence acquisition when decomposition fails. Together, these mechanisms enhance reasoning accuracy, stability, and efficiency.

3.1 Tree Structure Description

We categorize nodes based on their functional roles and behavioral dynamics during recursive query processing. These node types not only determine how the decomposition tree is expanded during top-down hierarchical reasoning, but also govern how results are aggregated during bottom-up inference. We define two primary node types as follows:

3.1.1 Query Node. Solving a complex multi-hop question involves a sequence of reasoning steps, where each step poses a sub-question conditioned on the information obtained from earlier answers. In this paper, the sub-questions correspond to the sub-queries that guide the retrieval and reasoning. Thus, we model the overall reasoning process as a query decomposition tree, where each node N acts as an execution unit for a specific sub-task. This structure allows the system to modularize complex questions into smaller components. Formally, a query node is defined as:

$$N_q = (q, d, a), \quad (1)$$

where q is the sub-query associated with the current node, d is the relevant retrieved document set based on q , and a is the candidate generated answer using the provided context d .

Query nodes represent the fundamental abstraction in our tree and are responsible for handling both the initial input query and any intermediate sub-queries generated during decomposition. If the model produces a high-confidence answer after querying the decomposition module, the answer is stored in a , and this node becomes a leaf, halting further expansion. If the node is decomposed into sub-query nodes, each child is used to perform retrieval and reasoning, and the answer field is left empty until a bottom-up aggregation is performed. Similarly, if entity nodes are generated instead, the answer field is also deferred until backtracking.

3.1.2 Entity Node. Entity nodes handle fallback cases where a query is neither directly answerable nor further decomposable. In

such scenarios, the system abstracts the query into key semantic entities and uses them for targeted evidence retrieval. This abstraction enables LLM to continue reasoning over meaningful units even when a complete answer or decomposition is not viable. Formally, each entity node is defined as:

$$N_e = (e, d), \quad (2)$$

where e is a tuple of salient entities extracted from the parent query, and d is the set of documents retrieved using them.

Entity nodes are terminal by design and do not spawn further children. They enhance the flexibility and robustness of the query tree by separating entity-level evidence collection from higher-level reasoning, particularly in cases involving vague, ambiguous, or semantically entangled queries.

3.2 Query Decomposition Module

This module, denoted as $f_decompose(q, d, p)$, constitutes a core functional process of our framework. It determines, given the current query q , its retrieved context d , and the parent query p , whether the query should be directly answered, decomposed into simpler sub-queries, or abstracted into entities for fine-grained retrieval.

3.2.1 Adaptive Node Expansion Mechanism. In the proposed query decomposition module, the system employs a top-down hierarchical decision process for each input query to determine its evolution path within the decomposition tree dynamically. This mechanism consists of three progressive judgment layers: first, the system assesses whether the current query can generate a reliable answer directly based on the retrieved evidence and the parent query context. If the answer is clear and exhibits high confidence, the current node is labeled as type "answer", indicating that the branch has converged and requires no further expansion.

If the query cannot be answered directly, the system further evaluates its decomposability—whether it can be rewritten as two more fundamental sub-queries. If feasible, a "query"-type intermediate node is constructed, and the sub-queries are processed recursively. When the query is unanswerable and undecomposable, the system executes a semantic abstraction strategy by extracting key entities from the query and constructing an "entity"-type node, thus terminating the branch as a structured semantic unit.

Both "answer" and "query" nodes are considered query nodes, where "answer" specifically denotes that the query has been resolved. The overall function can be formally described as follows:

$$f_decompose(q, d, p) = \begin{cases} ("answer", A) & \text{if } Ans(q) \\ ("query", q_1, q_2) & \text{if } \neg Ans(q) \wedge Spl(q) \\ ("entity", e_1, e_2) & \text{if } \neg Ans(q) \wedge \neg Spl(q), \end{cases} \quad (3)$$

where A is the generated answer written to the node (if available), q_1, q_2 are auto-generated sub-queries, e_1, e_2 are entities extracted from q , and p is parent node, which provides semantic supervision to preserve goal alignment. $Ans(q)$ refers to the LLM's judgment as to whether the context retrieved for the current sub-query q is sufficient to support an answer to that sub-query. $Spl(q)$ refers to the LLM's judgment, based on the inherent logic of the problem, as to whether the problem can be decomposed into two sub-problems.

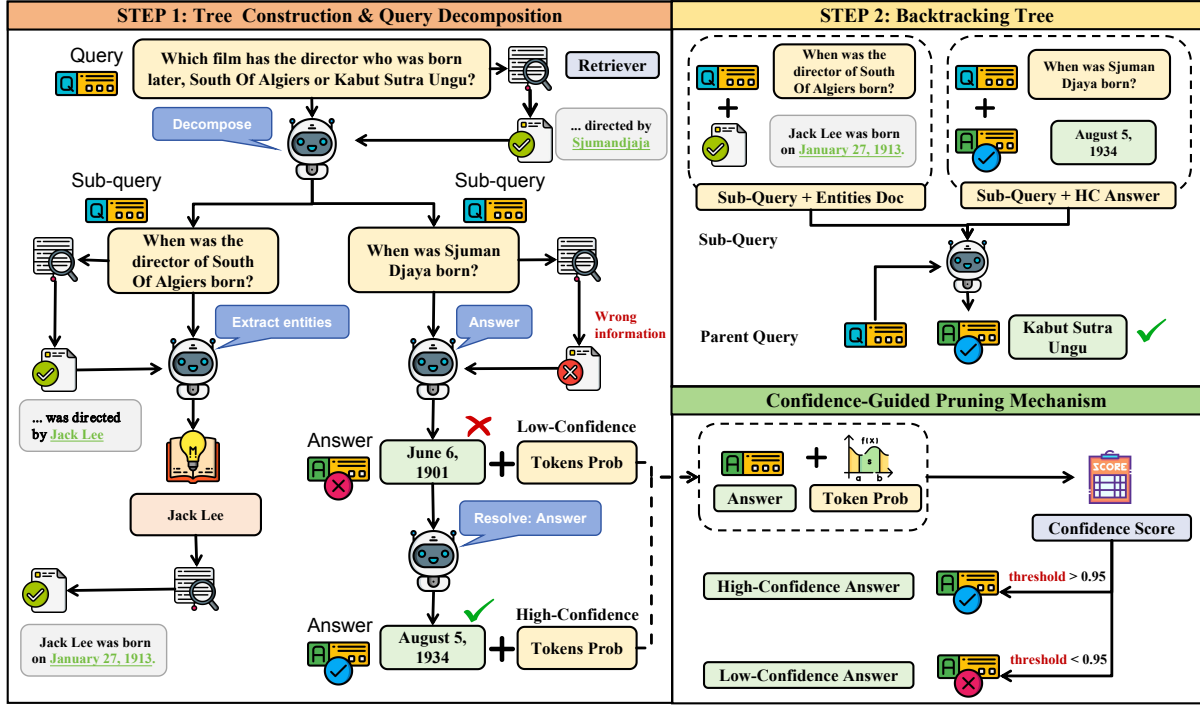


Figure 2: Overall framework of PruneRAG. The model constructs a query decomposition tree via confidence-guided expansion (Section 3.2.1) and pruning (Section 3.2.2), performs fine-grained retrieval when decomposition is infeasible (Section 3.2.3), and aggregates intermediate results through a bottom-up backtracking process to generate the final answer (Section 3.3).

3.2.2 Confidence-Guided Pruning Mechanism. To further mitigate the hallucination problem [27] that may arise when generating answer-type nodes, we design a confidence-guided decision mechanism based on the model’s prediction certainty. Specifically, if the adaptive node expansion mechanism generates a candidate answer A , the confidence-guided decision mechanism is triggered. This mechanism computes a confidence score based on the token-level log probability of the generated answer sequence and determines whether to accept the answer. In generative question answering, LLMs typically produce an answer sequence in an autoregressive manner, predicting one token at a time. Each token a_i is generated conditioned on the preceding tokens, the input query q , and the retrieved context d , following the probability distribution $P(a_i | a_{<i}, q, d)$. When the overall token-level log probability of the answer sequence is high, it indicates that the model is relatively confident in the generation process, semantically reliable, and syntactically coherent. Thus, we define answer confidence as:

$$\text{Confidence}(A) = \exp \left(\frac{1}{|A|} \sum_{i=1}^{|A|} \log P(a_i | a_{<i}, q, d) \right), \quad (4)$$

where $A = (a_1, a_2, \dots, a_{|A|})$ denotes the ordered sequential sequence of tokens constituting the answer; $P(a_i | a_{<i}, q, d)$ is the conditional likelihood probability of token a_i given the preceding generated tokens $a_{<i}$ along with the query and retrieved context.

This confidence score serves as a principled metric to guide node expansion decisions within the decomposition process, functioning as a pruning criterion to filter low-confidence branches early and

thus improve computational efficiency and answer quality. The operational workflow for each node $N_i = (q_i, d_i, a_i)$ is as follows:

- (1) Compute the confidence score of the candidate answer A as $\text{Confidence}(A)$;
- (2) If $\text{Confidence}(A)$ is greater than the threshold τ_A , the decision mechanism accepts the answer, meaning that the model can generate a high-confidence answer based on the current information. The node is then labeled as type "answer", A is written into the answer field of the node, and further expansion is stopped;
- (3) If $\text{Confidence}(A)$ is less than the threshold τ_A , the decision mechanism rejects the answer. The model then autonomously decides whether to continue decomposing the query into two sub-queries or to stop decomposition and extract key entities: if decomposable, a "query"-type node is created and q_1, q_2 are generated; if not, the fine-grained retrieval mechanism is triggered, an "entity"-type node is generated, entities are extracted, and this branch is terminated.

The confidence-guided decision mechanism takes the generation confidence of candidate answers as the core criterion, enhancing the system’s ability to assess answer quality. It plays a key role in controlling node convergence and preventing premature termination. By rejecting low-confidence answers, it effectively mitigates hallucinations, reduces invalid expansions, and improves the stability and efficiency of the query decomposition tree, thereby improving the reliability and controllability of multi-hop reasoning.

3.2.3 Fine-grained Retrieval Mechanism. When a query can no longer be decomposed, it often indicates its internal logical structure has been compressed or implicitly embedded. Further decomposition may compromise semantic integrity, introduce erroneous components, and reduce reasoning efficiency. Therefore, the system treats the query as the minimal semantic unit and terminates further decomposition. It then activates the fine-grained retrieval mechanism, which extracts key entities (such as person names, locations, and events) to construct structured retrieval anchors, guiding the system to locate relevant documents or passages within the knowledge corpus. For instance, given the query "Who is Danny Welch's employer?", it extracts "Danny Welch" and "Employer of Danny Welch". These entities are output as structured semantic information, labeling the node type as "entity". The entity set is used for retrieval, and the retrieved content is inserted into the node's context field d .

This mechanism enables precise information acquisition by treating entities as the smallest meaningful semantic unit when structural reasoning becomes ineffective. It preserves semantic completeness, significantly enhances retrieval specificity and accuracy, and stabilizes the overall reasoning process.

3.3 Bottom-Up Tree Backtracing

After the query decomposition tree is fully constructed, the system performs a systematic bottom-up backtracing process to progressively and coherently aggregate evidence and answers until the root node ultimately converges to the final reliable solution. We formally define the backtracing result of a node N as: $\text{Backtrace}(N) = f(N)$, where the function $f(N)$ behaves differently depending on the node type. Specifically: (i) if $N = (q, d, a)$ and the node already stores a high-confidence answer a , then $\text{Backtrace}(N) = a$; (ii) if $N = (q, d, \emptyset)$ is a query node without a direct answer, then $\text{Backtrace}(N)$ is obtained by aggregating the results of its children; (iii) if $N = (e, d)$ is an entity node, then $\text{Backtrace}(N)$ summarizes the retrieved documents d into supportive evidence.

3.4 PruneRAG Inference

Algorithm 1 outlines the inference process of PRUNERAG. Given a complex query q , the model first initializes the root of a query decomposition tree and places it into a processing queue.

In the top-down construction phase, nodes are expanded until the queue is exhausted. For each node, the system retrieves the top- k documents and prompts the LLM to generate a candidate answer along with its confidence. If the confidence exceeds the threshold τ_A , the node is marked as an answer leaf and its branch terminates. Otherwise, the node is either decomposed into sub-queries, which are added to the queue, or converted into an entity node when further decomposition is not feasible. This dynamic expansion enables confidence-guided pruning, effectively suppressing unreliable or redundant branches. Once the tree is constructed, a bottom-up backtracing phase is performed. Leaf nodes either return high-confidence answers or summarized evidence from entity retrieval. Internal nodes recursively aggregate the results of their children, preserving semantic dependencies across sub-queries. Finally, the root node integrates all collected information to output the final answer a^* .

Algorithm 1: PRUNERAG Inference

Input: Complex query q , Retriever $\mathcal{R}$, LLM $\mathcal{M}$, Document collection $\mathcal{D} = d_1, \dots, d_N$

Output: Final answer a^*

Step 1: Initialize Root Node

Create root node $N_q = (q, d = \emptyset, a = \emptyset)$ and push into queue Q .

Step 2: Top-Down Tree Construction

while Q not empty **do**

 Pop a node $N = (q, d, a)$ from Q ;

 Retrieve top- k documents d using $\mathcal{R}$ given q ;

$\mathcal{M}(q, d) \rightarrow (A, \text{Confidence}(A)(4))$;

if $\text{Confidence}(A) \geq \tau_A$ **then**

 Mark N as answer node with $a = A$; continue;

else if query q is decomposable **then**

 Create child nodes $\{q_1, q_2\}$ and push into Q ;

else

 Create entity node $N_e = (e, d)$ from extracted e ;

Step 3: Bottom-Up Backtracing

while there exist unprocessed internal nodes **do**

if leaf N is answer-node **then**

 return stored a to its parent

else if leaf N is entity-node **then**

 summarize retrieved docs d as evidence

else

 aggregate results of all child nodes and pass upward

Return the aggregated answer a^* at the root.

This procedure ensures efficient inference: pruning reduces unnecessary exploration, while the backtracing process guarantees that evidence is aggregated coherently across the reasoning path.

4 Experiments

4.1 Research Questions

In this section, we conduct a comprehensive experimental study to address the following research questions (RQs):

- **(RQ1)** Does evidence forgetting occur in multi-turn RAG, and can our method mitigate it to ensure reliable answer generation?
- **(RQ2)** Do the individual modules, especially the confidence mechanism, improve accuracy by alleviating evidence forgetting?
- **(RQ3)** Can our approach enhance accuracy while minimizing retrieval overhead via fewer, high-quality retrieval steps?
- **(RQ4)** How sensitive is our method to hyperparameters like confidence threshold and tree depth?

4.2 Experimental Setup

4.2.1 Datasets. We conducted experiments on four representative multi-hop question answering (QA) datasets: HotpotQA [48], 2WikiMultiHopQA (2WikiQA) [9], Musique [41], Bamboogle [24] and GPQA [26]. To compare with other QA tasks, we included two single-hop QA datasets, Natural Questions (NQ) [17] and TriviaQA[15]. Details can be found in the Appendix A.1.

Table 1: Comparison of answer accuracy, inference latency, and evidence forgetting rate across RAG methods on multi-hop QA datasets. Bold indicates the best result, and underline indicates the second-best. '-' denotes that the Evidence Forgetting Rate (EFR) is not applicable: for Vanilla, no retrieval documents are used; for RAG and MemoRAG, the denominator case where all golden documents are retrieved is zero. Inference latency and EFR are compared only among multi-turn retrieval methods.

Types	Methods	Time Cost (ms)	HotpotQA			2WikiQA			MusiQue		
			EM ↑	F1 ↑	EFR ↓	EM ↑	F1 ↑	EFR ↓	EM ↑	F1 ↑	EFR ↓
Llama-3.1-8B-Instruct											
No-Retrieval	Vanilla	56	24.2	30.1	-	19.4	24.8	-	5.4	9.1	-
Single-Retrieval	Standard RAG	153	49.0	55.1	27.6	23.2	29.9	47.5	7.2	11.5	-
	MemoRAG	8,566	31.2	38.8	53.1	23.0	28.7	68.1	7.8	12.3	-
Multi-Retrieval	React	1,500	24.2	27.1	65.2	15.0	19.4	73.0	7.8	11.6	68.4
	Search-o1	1,696	35.4	39.4	53.1	22.0	25.4	63.3	9.2	13.6	75.6
	Self-RAG	3,200	30.4	37.7	57.4	13.0	23.4	89.9	8.2	14.3	83.3
	ConTReGen	2,156	43.4	49.8	35.9	18.6	23.5	56.4	10.4	14.6	51.6
	RAG-Star	8,216	45.0	54.6	51.3	30.0	35.2	74.7	13.0	18.9	78.5
	ProbTree	3,838	32.2	39.3	54.4	27.8	32.6	62.6	11.0	17.1	91.6
	PruneRAG (Ours)	474	52.8	60.6 (5.5↑)	32.8	33.0	40.2 (5.0↑)	51.4	15.4	22.9 (4.0↑)	52.0
Qwen3-8B											
No-Retrieval	Vanilla	712	26.2	34.1	-	31.2	36.1	-	9.0	15.3	-
Single-Retrieval	Standard RAG	824	49.0	55.6	24.8	23.2	27.6	27.5	10.8	16.4	-
	MemoRAG	10,321	39.4	50.9	35.1	22.8	32.1	65.9	9.0	16.1	-
Multi-Retrieval	React	4,497	30.0	36.7	46.4	23.6	28.7	54.2	10.2	15.7	69.5
	Search-o1	4,841	36.4	42.6	56.9	23.2	27.5	54.3	12.4	17.4	62.1
	Self-RAG	3,200	30.4	37.7	57.4	13.0	23.4	89.9	8.2	14.3	85.7
	ConTReGen	4,507	47.6	55.4	30.5	29.0	33.9	27.8	11.8	17.2	60.0
	RAG-Star	14,709	48.6	57.8	52.4	31.0	37.8	73.9	12.2	19.8	84.2
	ProbTree	3,300	22.6	26.4	50.0	15.8	19.2	50.6	5.0	7.6	84.3
	PruneRAG (Ours)	2,254	56.6	63.6 (5.8↑)	23.1	41.2	44.4 (6.6↑)	26.0	17.6	25.6 (5.8↑)	38.4

4.2.2 Evaluation Metrics. To evaluate the performance of our method, following previous works [20], we consider two dimensions: (i) *effectiveness*, measuring answer quality and coverage, and (ii) *efficiency*, capturing reasoning cost in time and resources. Effectiveness is measured by Exact Match (EM) and F1, which assess answer correctness at both strict and token-overlap levels. Efficiency is evaluated through Document Recall (Recall), reflecting whether the golden document is retrieved; Retrieval Number (RN), indicating how often the retriever is invoked; Time Cost (Time), measured as the average model inference time per question under a fixed max_tokens setting. Finally, to quantify cases where the model retrieves sufficient evidence yet fails to generate the correct answer, we define the *Evidence Forgetting Rate (EFR)* as:

$$EFR = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{G_i \subseteq d_i \wedge a_i \neq a_i^*\}. \quad (5)$$

where N denotes the total number of evaluation samples, G_i is the golden evidence set containing all documents required to answer the i -th question, d_i represents the set of documents retrieved by the model throughout all retrieval rounds for that question, a_i is the model's predicted answer, and a_i^* is the corresponding ground-truth answer. The indicator function $\mathbf{1}\{\cdot\}$ takes value 1 if the retrieved evidence fully covers the golden set ($G_i \subseteq d_i$) while the generated

answer is incorrect ($a_i \neq a_i^*$), and 0 otherwise. Thus, EFR quantifies the proportion of cases in which the model fails to utilize complete evidence effectively, reflecting the severity of evidence forgetting in retrieval-augmented generation.

4.2.3 Baselines. We compare PruneRAG with four chain-based RAG methods: React [50], Search-o1 [20], Self-RAG [1] and MemoRAG [25], as well as three tree-structured RAG methods: ConTRGen [27], RAG-Star [13] and ProbTree[2]. React integrates language model reasoning with retrieval actions in an interleaved reasoning-action process; Search-o1 incorporates a document reasoning module to summarize and integrate document knowledge into the reasoning process. Self-RAG employs reflection tokens to let the model assess retrieved content and perform adaptive, on-demand retrieval. ConTRGen explores multiple semantic facets of the query hierarchically via a tree; RAG-Star utilizes the Monte Carlo Tree Search algorithm to explore the optimal reasoning path.

4.2.4 Implementation Details. Following previous work [13], we conduct experiments with two models: Qwen-3-8B [47] and Llama-3.1-8B-Instruct [7]. For Self-RAG, which is fine-tuned on Llama-2-7B-HF [39], we use its official release. Following Karpukhin et al. [16], we use the full Wikipedia 2018 dump as the retrieval corpus. Our main experiments follow the retrieval setup of Jiang et al.

[13], employing FAISS [14] for indexing, BGE-large-en-v1.5 [45] as retriever, and retrieving top-5 documents for answer generation. For PruneRAG, we set the tree maximum branching factor to 2, the maximum tree depth to 3, and the confidence threshold to 0.95. To ensure reproducibility, decoding is performed via greedy search with temperature 0. Experiments are conducted on a server with four NVIDIA L40 PCIe 48GB GPUs, and all inference is conducted using the vLLM framework[18, 22]. Further details can be found in the Appendix A.2.

4.3 Main Result (RQ1)

To investigate whether evidence forgetting is a prevalent issue in RAG, we compare representative methods under three retrieval paradigms: no retrieval, single-turn retrieval, and multi-turn retrieval. The results are presented in Table 1.

In the no-retrieval paradigm, the model answers questions purely from parametric knowledge, resulting in relatively low accuracy. The Evidence Forgetting Rate (EFR) is not applicable in this paradigm, since no external documents are retrieved. Under the single-turn retrieval paradigm, external knowledge is incorporated through a single retrieval and generation step. While this paradigm improves accuracy, it does not involve iterative reasoning, and thus, the phenomenon of evidence forgetting is relatively mild.

In the multi-turn retrieval paradigm, iterative methods consistently exhibit high EFR across datasets, confirming that evidence forgetting is a common weakness of this paradigm. For instance, ReAct, Search-o1, Self-RAG, RAG-Star, and ProbTree all exceed 46% EFR on HotpotQA, rise above 50% on 2WikiQA, and surpass 68% on Musique, despite the fact that golden documents have been fully retrieved. Importantly, these trends are consistent across different backbone models, indicating that evidence forgetting is a general weakness of multi-turn retrieval, independent of the base model. An exception is ConTReGen, which achieves a lighter degree of forgetting (35.9% and 30.5% on HotpotQA), benefiting from its tree-structured design that compresses reasoning depth. However, such mitigation remains limited, as the improvement in EFR does not translate into optimal accuracy. In contrast, PruneRAG consistently achieves the best or second-best EFR across all datasets under both backbone models, with values such as 23.1% on HotpotQA, 26.0% on 2WikiQA, and 38.4% on Musique (all reported under the Qwen-3-8B backbone). Beyond reducing forgetting, PruneRAG also delivers the highest EM and F1 scores, with improvements in accuracy arising from reduced evidence forgetting. At the same time, PruneRAG achieves the fastest inference speed among multi-turn RAG methods, owing to its confidence-guided pruning mechanism that effectively reduces redundant and wrong expansions, and on average runs 4.9× faster than mainstream multi-retrieval baselines.

4.4 Ablation Study (RQ2)

To investigate RQ2, we conduct an ablation study to assess the contribution of each core component in PruneRAG. Four variants are designed: *w/o Confidence*, which removes the confidence-guided pruning mechanism; *w/o Answer*, which removes the answer branch in adaptive node expansion; *w/o Entity*, which removes the fine-grained retrieval mechanism; and *w/o Adaptive*, which eliminates

Table 2: Ablation results of PruneRAG under the Qwen3-8B backbone. Bold denotes the best results. RN indicates average retrieval number (lower is better).

Method	HotpotQA			2WikiQA			Musique		
	EM	RN	EFR	EM	RN	EFR	EM	RN	EFR
PruneRAG	56.6	2.0	23.1	41.2	3.4	26.0	17.6	3.3	38.4
w/o Con.	53.4	1.9	25.1	39.6	3.3	26.2	16.2	3.2	51.2
w/o Ans.	37.8	3.3	48.2	33.2	4.4	55.3	15.0	4.8	65.0
w/o Ent.	53.4	1.7	24.4	40.0	2.8	27.8	15.4	3.1	53.3
w/o Ada.	42.0	5.3	43.2	32.0	4.6	50.0	15.2	5.8	67.7

Table 3: Comparison of different RAG methods in terms of average Retrieval Numbers (RN), document recall rates, and Retrieval Efficiency (RE).

Method	HotpotQA		2Wiki		Musique		RE
	RN	Recall	RN	Recall	RN	Recall	
React	2.61	47.5	3.25	47.0	2.83	20.0	13.2
Search-o1	3.09	54.3	4.35	53.8	3.68	25.8	12.3
Self-RAG	1.89	54.3	1.96	32.4	1.92	9.7	16.7
ConTReGen	3.76	58.0	2.81	36.3	3.87	13.3	10.5
RAG-Star	6.73	42.9	4.30	29.4	3.49	11.4	5.4
ProbTree	4.00	61.7	4.25	42.0	4.14	22.4	10.2
PruneRAG	2.06	60.7	3.40	47.4	3.39	23.2	16.7

the adaptive expansion strategy, decomposing queries unconditionally. As shown in Table 2, removing any of the four modules leads to clear performance degradation, indicating that each design is indispensable for the overall effectiveness of PruneRAG. (a) **w/o Confidence**. Eliminating the confidence-guided pruning mechanism leads to a increase in EFR, reflecting the model’s tendency to accept unreliable outputs and disrupt critical evidence transmission. Importantly, both *w/o Answer* and *w/o Adaptive* also remove this pruning mechanism, and their EFR values exhibit a similar escalation, consistent with *w/o Confidence*. These results collectively underscore that confidence-based pruning is indispensable for mitigating evidence forgetting and for sustaining reliable answer accuracy. (b) **w/o Adaptive**. Eliminating adaptive control produces the most pronounced declines in most datasets, as uncontrolled decomposition generates redundant sub-queries and unstable reasoning structures. This underscores that adaptive expansion is essential for confidence-guided pruning. (c) **w/o Answer**. Removing the answer branch prevents the model from using intermediate conclusions, which destabilizes reasoning and increases retrieval calls, thus raising inference cost. The presence of this module reduces retrieval frequency and supports more efficient reasoning. (d) **w/o Entity**. Without entity-level retrieval, leaf nodes terminate prematurely, resulting in incomplete evidence utilization and increased forgetting. This weakens semantic coverage and lowers accuracy.

4.5 Retrieval Efficiency Analysis (RQ3)

Table 3 compares document recall (Recall), the retrieval number (RN) and Retrieval Efficiency (RE) across different methods. RE

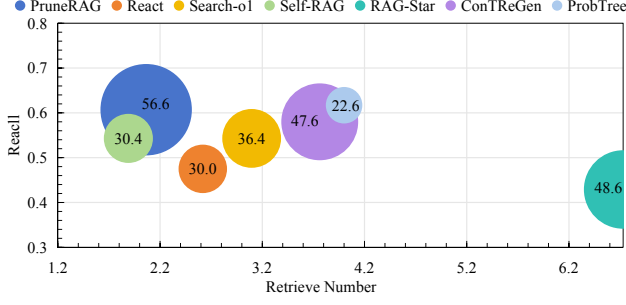


Figure 3: Retrieval efficiency on HotpotQA, where the x-axis denotes retrieval count, the y-axis denotes golden document recall rate, and bubble size indicates EM.

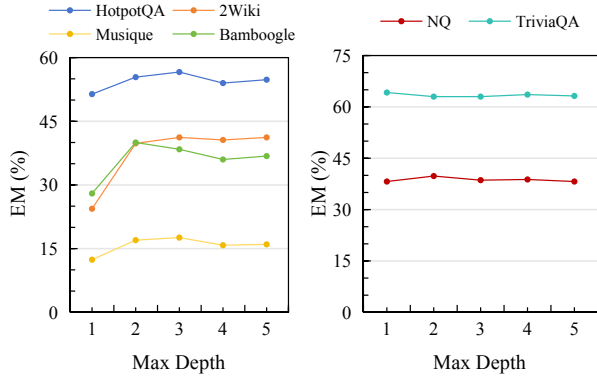


Figure 4: Impact of the maximum query tree depth on performance across datasets with varying reasoning complexity.

refers to the ratio between document recall (Recall) and the retrieval number (RN). The results show that PruneRAG consistently achieves higher recall with fewer retrieval calls. This advantage stems from dynamic query decomposition, where sub-queries are generated based on the parent query and its retrieved documents, and redundant sub-queries are suppressed once the corresponding evidence has already been obtained. Moreover, the fine-grained retrieval mechanism further enhances recall: when a query can no longer be decomposed or directly answered, key entities with reduced noise are extracted for targeted retrieval, improving the likelihood of capturing relevant evidence.

Figure 3 further illustrates the trade-off between retrieval count, recall rate, and accuracy on HotpotQA. While baseline methods often rely on increasing retrievals to boost recall, this strategy introduces redundant or distracting information and fails to translate into accuracy gains. In contrast, PruneRAG achieves higher EM scores even at comparable or lower recall levels, demonstrating a superior ability to exploit retrieved golden documents effectively.

4.6 Hyper-parameters Analysis (RQ4)

To address RQ4, we analyze the sensitivity of PruneRAG to two key hyperparameters: the maximum query tree depth and the confidence threshold τ_A .

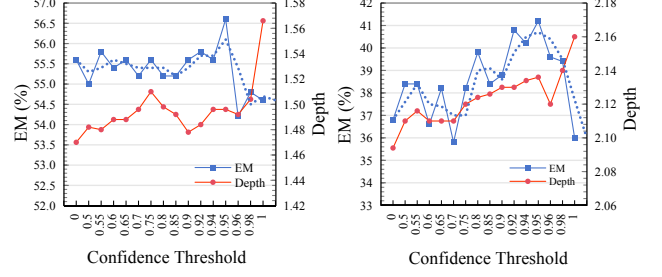


Figure 5: Impact of confidence threshold τ_A on answer accuracy (Exact Match) and average query tree depth across HotpotQA (left) and 2WikiQA (right). The results demonstrate that the threshold range $\tau_A \in [0.9, 0.95]$ consistently achieves strong performance.

Maximum Tree Depth. As illustrated in Figure 4, PruneRAG demonstrates a strong ability to adapt reasoning depth to task complexity. On multi-hop QA datasets such as HotpotQA, performance steadily improves as the maximum depth increases, reaching an optimal point around depth 2–3 before saturating. This pattern aligns with the typical reasoning steps required for most multi-hop QA instances, indicating that the model effectively decomposes complex questions and integrates cross-paragraph information. In contrast, for predominantly single-hop datasets such as NQ and TriviaQA, the best performance is observed when the depth is set to 1, and deeper expansions introduce noise that slightly impairs accuracy. These results highlight that PruneRAG can perceive task complexity and dynamically adjust reasoning depth accordingly.

Confidence Threshold τ_A . Figure 5 shows the impact of τ_A on answer accuracy and average tree depth. When $\tau_A < 0.5$, the model tends to accept answers prematurely, producing shallow reasoning structures and unstable accuracy. In the mid-to-high range ($\tau_A \in [0.9, 0.95]$), the confidence-guided pruning mechanism is fully activated, rejecting more low-quality answer nodes and thereby yielding deeper reasoning paths and more reliable answers. On HotpotQA and 2WikiQA, peak performance is achieved at $\tau_A = 0.95$, with EM scores of 56.6% and 41.2% and corresponding depths of 1.49 and 2.13, respectively.

5 Conclusion

In this paper, we propose PruneRAG, a confidence-guided query decomposition tree framework to address the twin challenges of evidence forgetting and inefficiency in retrieval-augmented generation. By combining adaptive expansion, confidence-guided pruning, and fine-grained entity retrieval, PruneRAG preserves key evidence while reducing redundant reasoning. Experiments on various multi-hop QA benchmarks demonstrate consistent gains in both accuracy and efficiency, with significant mitigation of evidence forgetting.

Acknowledgments

This Work is supported by NSFC (No.62372139), NSF of Guangdong (No. 2024A1515030024), Research Projects of Shenzhen (No. ZDCY20250901100302003), part of idea is inspired by perspectives shared at the CCF YOCSEF Shenzhen "Large Model Inference" series forum (No. CCF-Yo-25-111).

References

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *ICLR*. OpenReview.net.
- [2] Shulin Cao, Jiajie Zhang, Jiaxin Shi, Xin Lv, Zijun Yao, Qi Tian, Lei Hou, and Juanzi Li. 2023. Probabilistic Tree-of-thought Reasoning for Answering Knowledge-intensive Complex Questions. In *EMNLP (Findings)*. Association for Computational Linguistics, 12541–12560.
- [3] Qiguang Chen, Libo Qin, Jinhao Liu, Dengyun Peng, Jiannan Guan, Peng Wang, Mengkang Hu, Yuhang Zhou, Te Gao, and Wanxiang Che. 2025. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *arXiv preprint arXiv:2503.09567* (2025).
- [4] Wenfeng Feng, Chuzhan Hao, Yuewei Zhang, Jingyi Song, and Hao Wang. 2025. AirRAG: Activating Intrinsic Reasoning for Retrieval Augmented Generation using Tree-based Search. *arXiv:2501.10053 [cs.AI]*
- [5] Ruiliu Fu, Han Wang, Xuejun Zhang, Jun Zhou, and Yonghong Yan. 2021. Decomposing Complex Questions Makes Multi-Hop QA Easier and More Interpretable. In *Findings of the Association for Computational Linguistics: EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 16-20 November, 2021*. Association for Computational Linguistics, 169–180.
- [6] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv:2312.10997 [cs.CL]*
- [7] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, and et al. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783 [cs.AI]*
- [8] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*. PMLR, 3929–3938.
- [9] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020*. International Committee on Computational Linguistics, 6609–6625.
- [10] Chengkai Huang, Yu Xia, Rui Wang, Kaige Xie, Tong Yu, Julian McAuley, and Lina Yao. 2025. Embedding-informed adaptive retrieval-augmented generation of large language models. In *Proceedings of the 31st International Conference on Computational Linguistics*. 1403–1412.
- [11] Chengkai Huang, Tong Yu, Kaige Xie, Shuai Zhang, Lina Yao, and Julian McAuley. 2024. Foundation models for recommender systems: A survey and new perspectives. *arXiv preprint arXiv:2402.11143* (2024).
- [12] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*. Association for Computational Linguistics, 7036–7050.
- [13] Jinhao Jiang, Jiayi Chen, Junyi Li, Ruiyang Ren, Shijie Wang, Xin Zhao, Yang Song, and Tao Zhang. 2025. RAG-Star: Enhancing Deliberative Reasoning with Retrieval Augmented Verification and Refinement. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Albuquerque, New Mexico, 7064–7074.
- [14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [15] Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*. Association for Computational Linguistics, 1601–1611.
- [16] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *EMNLP (1)*. 6769–6781.
- [17] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur P. Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: a Benchmark for Question Answering Research. *Trans. Assoc. Comput. Linguistics* 7 (2019), 452–466.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*. ACM, 611–626.
- [19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [20] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. 2025. Search-o1: Agentic search-enhanced large reasoning models. *arXiv preprint arXiv:2501.05366* (2025).
- [21] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *arXiv:2307.03172 [cs.CL]*
- [22] Yuanfeng Meng, Xin Jin, Lina Lei, Chun-Le Guo, and Chongyi Li. 2025. UltraLED: Learning to See Everything in Ultra-High Dynamic Range Scenes. *CoRR* abs/2510.07741 (2025).
- [23] Xianshu Peng and Wei Wei. 2025. GRAT: Guiding Retrieval-Augmented Reasoning through Process Rewards Tree Search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*. Association for Computational Linguistics, 27861–27875.
- [24] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2023. Measuring and Narrowing the Compositionality Gap in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*. Association for Computational Linguistics, 5687–5711.
- [25] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. 2025. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. In *WWW*. ACM, 2366–2377.
- [26] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *CoRR* abs/2311.12022 (2023).
- [27] Kashob Kumar Roy, Pritom Saha Akash, Kevin Chen-Chuan Chang, and Lucian Popa. 2024. ConTReGen: Context-driven Tree-structured Retrieval for Open-domain Long-form Text Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Miami, Florida, USA, 13773–13784.
- [28] Alireza Salemi and Hamed Zamani. 2024. Evaluating Retrieval Quality in Retrieval-Augmented Generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14-18, 2024*. ACM, 2395–2400.
- [29] Michael Shen, Muhammad Umar, Kiwan Maeng, G. Edward Suh, and Udit Gupta. 2024. Towards Understanding Systems Trade-offs in Retrieval-Augmented Generation Model Inference. *arXiv:2412.11854 [cs.AR]*
- [30] Tiesunlong Shen, Jin Wang, Xuejie Zhang, and Erik Cambria. 2025. Reasoning with Trees: Faithful Question Answering over Knowledge Graph. In *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*. Association for Computational Linguistics, 3138–3157.
- [31] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval Augmentation Reduces Hallucination in Conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 16-20 November, 2021*. Association for Computational Linguistics, 3784–3803.
- [32] Ishneet Sukhvinder Singh, Ritvik Aggarwal, Ibrahim Allahverdiyev, Muhammad Taha, Aslihan Akalin, Kevin Zhu, and Sean O'Brien. 2025. ChunkRAG: Novel LLM-Chunk Filtering Method for RAG Systems. *arXiv:2410.19572 [cs.CL]*
- [33] Weihang Su, Yichen Tang, Qingyao Ai, Zhijiang Wu, and Yiqun Liu. 2024. DRAGIN: Dynamic Retrieval Augmented Generation based on the Real-time Information Needs of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*. Association for Computational Linguistics, 12991–13013.
- [34] Hao Sun, Hengyi Cai, Yuchen Li, Xuanbo Fan, Xiaochi Wei, Shuaiqiang Wang, Yan Zhang, and Dawei Yin. 2025. Enhancing Retrieval-Augmented Generation via Evidence Tree Search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*. Association for Computational Linguistics, 24116–24127.
- [35] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-graph: Knowledge graph empowered large language model reasoning. In *Proceedings of the ACM on Web Conference 2025*. 3505–3522.
- [36] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2025. Hydrarag: Structured cross-source enhanced large language model reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 14442–14470.
- [37] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2025. MemoTime: Memory-Augmented Temporal Knowledge Graph Enhanced Large Language Model Reasoning. *arXiv preprint arXiv:2510.13614* (2025).
- [38] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2026. PrivGemo: Privacy-Preserving Dual-Tower Graph Retrieval for Empowering LLM Reasoning with Memory Augmentation. *arXiv preprint arXiv:2601.08739* (2026).

- [39] Hugo Touvron, Louis Martin, Kevin Stone, and et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023).
- [40] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *arXiv preprint arXiv:2212.10509* (2022).
- [41] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Trans. Assoc. Comput. Linguistics* 10 (2022), 539–554.
- [42] Liang Wang, Nan Yang, Xiaolong Huang, and et al. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *CoRR* abs/2212.03533 (2022). arXiv:2212.03533
- [43] Shang Wang, Tianqing Zhu, Dayong Ye, and Wanlei Zhou. 2024. When Machine Unlearning Meets Retrieval-Augmented Generation (RAG): Keep Secret or Forget Knowledge? arXiv:2410.15267 [cs.CR]
- [44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [45] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. *CoRR* abs/2309.07597 (2023). arXiv:2309.07597
- [46] Shicheng Xu, Liang Pang, Huawei Shen, Xueqi Cheng, and Tat-Seng Chua. 2024. Search-in-the-chain: Interactively enhancing large language models with search for knowledge-intensive tasks. In *Proceedings of the ACM Web Conference 2024*. 1362–1373.
- [47] An Yang, Anfeng Li, Baosong Yang, and et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL]
- [48] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*. Association for Computational Linguistics, 2369–2380.
- [49] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- [51] Xiangjun Zai, Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, and Wenjie Zhang. 2025. PROH: Dynamic Planning and Reasoning over Knowledge Hypergraphs for Retrieval-Augmented Generation. *arXiv preprint arXiv:2510.12434* (2025).

A Appendix

A.1 Datasets

We evaluate our method on six QA datasets of varying complexity. Among them, four datasets feature complex, multi-hop reasoning challenges such as cross-passage evidence integration, factual alignment, entity linking, and logical composition:

- **HotpotQA** [48]: A widely used multi-hop QA benchmark where each question requires integrating evidence from multiple documents. We follow the full wiki setting.
- **2WikiMultihopQA** [9]: Constructed from Wikipedia with explicitly structured multi-hop questions, making it well-suited for evaluating path-based and compositional reasoning.
- **MuSiQue-full** [41]: Composed of multiple independently meaningful sub-questions, naturally aligned with query-tree style decomposition and testing information integration ability.
- **BAMBOOGLE** [24]: A challenging dataset emphasizing instruction-style QA, with frequent intent shifts and cross-sentence constraints that demand structured reasoning.
- **GPQA** [26]: A graduate-level benchmark designed to stress-test compositional reasoning and domain knowledge.

To verify the adaptive capability of PruneRAG across varying levels of reasoning complexity, we include two single-hop QA datasets:

- **Natural Questions (NQ)** [17]: Real search queries paired with Wikipedia passages, with most questions answerable from a single paragraph, serving as a factual QA baseline.
- **TriviaQA** [15]: Trivia-style questions with short entity answers and minimal reasoning, suitable for evaluating fact retrieval.

Following Jiang et al. [13], we randomly sample 500 validation instances as the test set for each dataset, except for BAMBOOGLE, where all 125 available instances are used. Notably, only HotpotQA, 2WikiMultihopQA, and MuSiQue provide annotated golden documents, enabling the computation of Evidence Forgetting Rate (EFR). For the remaining datasets, evaluation focuses on accuracy and efficiency metrics, ensuring comprehensive assessment under both golden-evidence and open-domain settings.

A.2 Implementation Details

In all experiments, we adopt large language models (LLMs) as the backbone generators and evaluate the performance of various methods under different retrieval strategies. Specifically, we select two mainstream open-source models as base models: **LLaMA3.1-8B-Instruct** [7] and **Qwen3-8B** [47]. Since the Self-RAG method is fine-tuned on the LLaMA2-7b-HF [39] model, we directly use the released checkpoint provided by the original authors to ensure fairness and comparability. To further assess the generalizability of our method across models of different parameter scales, we additionally evaluate on two larger models: **LLaMA3.1-70B-Instruct** and **Qwen3-32B**.

For the retriever component, we evaluate two widely used dense retrieval strategies: **BGE** [45] and **E5** [42]. In the main experiments, following the RAG-Star setup, we use **BGE-large-en-v1.5** [45] as the default retriever. Each sub-query retrieves the top-5 documents, which are used to construct the retrieval-augmented context. For the knowledge corpus, we adopt the 2018 English Wikipedia dump used in DPR [16], and build dense indexes through a unified pre-processing pipeline to ensure consistency across retrievers. Results based on E5 are included in the Appendix A.5.

For PruneRAG, we set the tree branching factor to 2, maximum depth to 3, and the confidence threshold τ_A to 0.95. All hyperparameter settings for baseline methods follow their original implementations. During inference, we set max_tokens to 4096 for all models. For Qwen-based models, we observe frequent token repetition and apply the official remedy by setting repetition_penalty to 1.05 to mitigate this issue.

Considering that most baselines adopt multi-step querying strategies, for fair comparison, we cap the number of retriever calls to 7 for all methods to ensure a fair comparison. This limit corresponds to the configuration of PruneRAG with depth 3, where a full ternary tree contains up to 7 nodes.

All experiments are conducted on a server equipped with 4× **NVIDIA L40 PCIe 48GB GPUs**, and inference is performed using the **vLLM** framework [18, 22].

Table 4: Comparison of answer accuracy (EM/F1) and inference latency across RAG methods on higher-complexity datasets (Bamboogle and GPQA). Bold indicates the best result, and underline indicates the second-best. Inference latency is compared only among multi-turn retrieval methods.

Methods	Time Cost (ms)	Bamboogle		GPQA	
		EM ↑	F1 ↑	EM ↑	F1 ↑
Llama-3.1-8B-Instruct					
Vanilla	153	12.0	19.7	24.7	24.7
Standard RAG	813	24.0	32.5	24.2	24.2
MemoRAG	10,405	17.6	27.2	20.2	20.3
React	4,637	14.4	18.9	0.5	0.6
Search-o1	4,024	19.2	23.7	6.0	6.3
Self-RAG	3,634	7.2	15.7	3.0	3.6
ConTRGen	6,022	24.0	31.5	14.1	14.5
RAG-Star	31,253	19.2	24.9	11.1	11.5
ProbTree	4,035	24.8	34.1	13.6	13.7
PruneRAG (Ours)	855	26.4	38.8	29.7	29.7
Qwen3-8B					
Vanilla	3,433	33.6	45.3	47.4	47.4
Standard RAG	3,673	31.2	38.1	50.5	50.5
MemoRAG	33,966	17.6	27.8	35.8	35.8
React	10,872	14.4	18.4	7.5	8.2
Search-o1	9,527	24.8	29.2	30.3	30.3
Self-RAG	3,634	7.2	15.7	3.0	3.6
ConTRGen	18,830	30.4	39.2	51.0	51.0
RAG-Star	12,822	32.8	44.9	47.9	47.9
ProbTree	9,304	16.8	22.4	37.3	37.3
PruneRAG (Ours)	8,895	34.4	46.7	53.5	53.5

A.3 Evaluation on Higher-Complexity Datasets

To further validate robustness, we evaluate PruneRAG on two more complex datasets, Bamboogle and GPQA. As shown in Table 4, baseline methods generally show substantial degradation on these tasks, highlighting the challenge of retrieving and reasoning over long-tail knowledge. In contrast, PruneRAG consistently outperforms all alternatives in EM and F1 under both LLaMA-3.1-8B and Qwen3-8B. For instance, it achieves 26.4% EM and 38.8% F1 on Bamboogle with LLaMA-3.1-8B, and 53.5% EM and 53.5% F1 on GPQA with Qwen3-8B, surpassing the strongest baselines while maintaining lower inference cost. Since these datasets do not provide golden evidence annotations, the Evidence Forgetting Rate (EFR) cannot be computed. Overall, the results confirm that the mechanisms of confidence-guided pruning and dynamic decomposition remain effective even under higher reasoning complexity.

A.4 Evaluation with Larger-Scale Base Models

We further investigate whether increasing the scale of backbone models can inherently alleviate evidence forgetting. Table 5 presents results on HotpotQA using LLaMA-3.1-70B-Instruct and Qwen3-32B. Although larger models generally achieve stronger accuracy

Table 5: Performance Comparison of PruneRAG and Baselines on Large-Scale Language Models on HotpotQA

Methods	Llama-3.1-70B-Instruct				Qwen3-32B			
	Time	EM	F1	EFR	Time	EM	F1	EFR
Vanilla	515	25.0	30.8	-	3,459	21.2	25.4	-
RAG	603	<u>50.6</u>	57.2	<u>29.7</u>	1,394	<u>54.2</u>	<u>62.6</u>	21.9
MemoRAG	1,000	10.0	10.0	10.0	1,000	10.0	10.0	10.0
React	2,824	41.2	46.6	41.8	7,927	34.4	40.4	35.5
Search-o1	3,057	46.2	52.6	40.2	5,047	42.4	49.4	43.6
Self-RAG	5,294	30.6	37.5	59.1	5,294	30.6	37.5	59.1
ConTRGen	22,799	50.6	<u>57.7</u>	30.7	16,921	49.8	58.7	36.4
RAG-Star	43,239	41.6	50.9	49.6	52,527	38.4	46.9	49.3
ProbTree	2,583	36.4	44.6	50.5	<u>2,504</u>	25.6	38.1	59.0
PruneRAG	1,432	58.2	66.0	29.2	2,221	58.4	66.0	<u>24.6</u>

than their smaller counterparts, the phenomenon of evidence forgetting remains evident: multi-turn baselines such as ReAct, Search-o1, and RAG-Star still exhibit high EFR values (often exceeding 40%), and even tree-structured methods such as ConTRGen and ProbTree do not fully overcome this limitation.

In contrast, PruneRAG consistently achieves the best overall performance across both backbones. On LLaMA-3.1-70B, it reaches 58.2% EM and 66.0% F1 while reducing EFR to 29.2%. Similarly, on Qwen3-32B, it achieves 58.4% EM and 66.0% F1 with an EFR of 24.6%, ranking best or second-best across all metrics. These improvements highlight that confidence-guided pruning and adaptive decomposition remain indispensable, even when the underlying language model has substantially larger capacity.

Overall, these findings demonstrate that scaling up model size alone does not resolve the challenge of evidence forgetting. Instead, principled mechanisms such as those introduced in PruneRAG are crucial for ensuring reliable reasoning and maximizing the utility of retrieved evidence in large-scale settings.

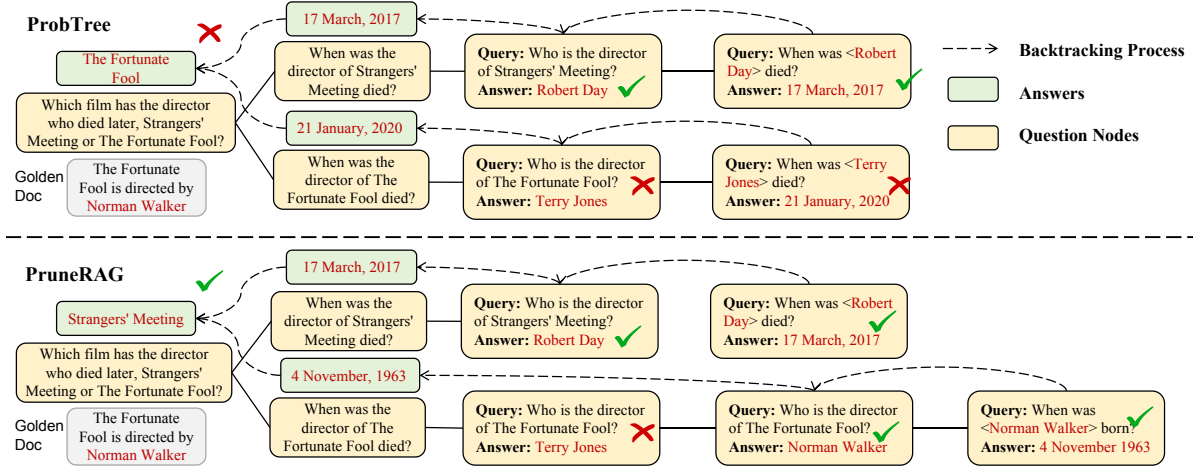
A.5 Evaluation with Alternative Retriever

To further examine the robustness of our framework, we replace the default retriever with the E5 embedding model and re-evaluate on three representative datasets. As shown in Table 6, evidence forgetting remains a prominent issue across baselines: multi-turn methods such as ReAct, Search-o1, and RAG-Star continue to suffer from high EFR values, often above 48%, despite the use of a different retriever. These results indicate that the forgetting phenomenon is not solely attributable to retriever quality, but persists as a systematic weakness of iterative retrieval paradigms.

In contrast, PruneRAG consistently delivers superior performance. Under both LLaMA-3.1-8B and Qwen3-8B backbones, it achieves the highest EM and F1 scores on HotpotQA, 2WikiQA, and MusiQue, while also maintaining lower or second-lowest EFR values. For example, on HotpotQA, PruneRAG attains 52.2% EM and 59.5% F1 with an EFR of 29.9% under Qwen3-8B, outperforming all multi-turn baselines by a clear margin. At the same time, it achieves these improvements with substantially reduced inference latency compared to other multi-turn methods.

Table 6: Performance of RAG methods using the E5 retriever on multi-hop QA datasets. Inference latency and EFR are compared only among multi-turn retrieval methods.

Types	Methods	Time Cost (ms)	HotpotQA			2WikiQA			MusiQue		
			EM ↑	F1 ↑	EFR ↓	EM ↑	F1 ↑	EFR ↓	EM ↑	F1 ↑	EFR ↓
Llama-3.1-8B-Instruct											
No-Retrieval	Vanilla	64	25.6	30.8	-	18.6	25.0	-	5.2	9.5	-
Single-Retrieval	Standard RAG	277	44.8	51.3	30.5	17.0	22.5	35.7	7.0	11.0	-
	MemoRAG	8,562	31.8	39.1	58.1	19.8	25.9	74.4	8.0	11.7	-
Multi-Retrieval	React	3,144	23.4	27.0	60.0	17.4	21.7	70.0	8.0	12.0	70.3
	Search-o1	2,278	33.4	37.6	55.9	22.6	27.6	64.8	12.6	16.7	58.1
	Self-RAG	3,107	29.0	36.0	59.2	15.6	26.4	83.3	6.8	13.0	83.3
	ConTReGen	1,864	40.2	46.5	41.0	19.2	24.0	42.8	7.8	10.8	80.0
	RAG-Star	8,265	42.0	54.4	49.7	34.0	42.0	67.5	10.2	14.7	88.4
	ProbTree	3,718	31.0	38.3	51.7	31.6	36.6	49.3	9.4	14.5	80.0
	PruneRAG (Ours)	746	48.6	56.5	30.5	35.8	42.8	46.3	13.8	22.4	55.6
Qwen3-8B											
No-Retrieval	Vanilla	740	27.2	36.2	-	30.8	35.6	-	10.2	16.2	-
Single-Retrieval	Standard RAG	1,221	46.6	53.8	30.5	24.0	28.3	23.8	10.8	15.6	-
	MemoRAG	10,669	35.6	46.5	50.0	22.6	30.9	41.8	8.6	13.9	-
Multi-Retrieval	React	7,192	30.0	36.2	48.7	25.0	30.0	53.5	11.2	15.8	50.0
	Search-o1	8,296	34.2	40.9	54.6	26.4	31.6	48.0	13.4	19.8	62.7
	Self-RAG	3,107	29.0	36.0	59.2	15.6	26.4	83.3	6.8	13.0	83.3
	ConTReGen	4,897	44.8	52.5	36.0	27.4	31.6	30.3	11.6	16.7	50.0
	RAG-Star	10,141	45.2	51.7	55.4	37.2	42.5	65.1	12.6	20.1	72.7
	ProbTree	3,633	33.6	40.3	49.3	27.6	31.1	51.9	5.2	10.9	85.7
	PruneRAG (Ours)	2,849	52.2	59.5	29.9	40.2	46.0	27.7	16.0	23.6	40.7

**Figure 6: Case study: ProbTree suffers from evidence forgetting, while PruneRAG prunes errors and recovers the correct reasoning chain.**

A.6 Case Study

Figure 6 illustrates an example of evidence forgetting when comparing the directors of *Strangers' Meeting* and *The Fortunate Fool*. ProbTree first retrieves the correct evidence that *The Fortunate Fool* was directed by Norman Walker, but incorrectly accepts "Terry Jones" as an intermediate answer. Without pruning, this error propagates and the reasoning path diverges, causing the golden evidence

to be lost and the final answer to be wrong. In contrast, PruneRAG prunes the low-confidence branch and regenerates a sub-query consistent with the retrieved evidence, correctly recovering Norman Walker. This allows the reasoning chain to integrate the true death dates and reach the correct conclusion. The case illustrates how PruneRAG suppresses low-quality outputs, preserves key evidence, and maintains robust multi-hop reasoning.